UniThrift — API Documentation (Final Version)

Base URL: http://localhost:5000/api

This document outlines the complete API routes built in the Node.js/Express backend across all three Sprints. These endpoints enable the React frontend to communicate with the MS SQL Server database.

Note: Endpoints marked with 🔒 require the authenticateToken JWT middleware in the request headers: { Authorization: "Bearer <token>" }.

👤 1. Authentication & User Management

POST /signup

What it does: Registers a new student to the platform.

Database Action:

1. Checks the Users table to ensure the email doesn't already exist.
2. Validates the @nu.edu.pk domain.
3. Hashes the password using bcryptjs.
4. Saves the uploaded profile picture via multer (or assigns a local default).
5. Executes an INSERT query to add the new record into the Users table.

POST /login

What it does: Authenticates a returning student and generates a JWT.

Database Action:

1. Queries the Users table by email.
2. Compares the hashed password.
3. If credentials match, generates a signed JWT token valid for 24 hours and returns it along with user data.

🔒 PUT /users/update/:id

What it does: Saves edits made in the Profile Page modal.

Database Action:

1. Verifies JWT authenticity.
2. Runs a SELECT query to ensure the chosen name isn't taken (user_id != @id).
3. Executes an UPDATE query to save the new bio, department, and password (re-hashing if changed).

🛒 2. Marketplace & Discovery

GET /items

What it does: Fetches all item listings for the main grid.

Database Action: Retrieves items joined with Users and Departments. Filters to strictly show items where (stock_quantity > 0 OR is_digital = 1) and status = 'available'.

GET /items/:itemId

What it does: Retrieves the full, detailed view of a single listing.

Database Action: Fetches granular item data including seller stats (reliability_score), dynamically calculating the wishlist_count, item_sold_count, and item_rating via SQL aggregate subqueries.

🔒 POST /items/upload

What it does: Creates a brand new item listing (Physical or Digital).

Database Action: Intercepts multipart/form-data. Saves images to /uploads and E-books to /secure_vault. Executes INSERT INTO Items, flagging is_digital = 1 if a file is uploaded.

🔒 PUT /items/update/:itemId

What it does: Saves changes when a seller edits their own listing.

Database Action: Checks ownership, then executes an UPDATE query.

🔒 DELETE /items/:itemId

What it does: Soft-deletes a listing.

Database Action: Executes an UPDATE Items SET status = 'deleted' to preserve transaction histories instead of hard deletion.

📚 3. The Digital Vault

🔒 GET /vault/:txId

What it does: Streams an E-Book PDF directly to the borrower's browser.

Database Action:

1. Queries the Transactions table to verify the user owns a completed transaction for this digital item.
2. Checks that GETUTCDATE() hasn't surpassed the return_date.
3. If valid, uses Node's fs.createReadStream to securely pipe the PDF file without exposing

the raw file path.

🤝 4. Transaction Logistics & Handshakes

🔒 POST /transactions/borrow

What it does: Initiates a request to borrow an item.

Database Action: - **If Digital:** Bypasses standard locks, instantly writing a pending transaction and firing a notification for the seller to grant access.

- **If Physical:** Executes the strict sp_InitiateBorrowHandshake stored procedure, which temporarily deducts stock_quantity to prevent overlapping requests (the "Greedy Goblin" fix).

🔒 POST /transactions/handshake/resolve

What it does: Accepts or rejects a pending borrow handshake.

Database Action:

- Executes sp_ResolveHandshake.
- If accepted: Computes the return_date using GETUTCDATE() to prevent timezone shifts.
- If rejected: Restores the item's stock_quantity.

🔒 POST /transactions/return/initiate & POST /transactions/return/resolve

What it does: Manages the physical return lifecycle.

Database Action: The borrower initiates a return (firing a notification). The lender resolves it, triggering an UPDATE that sets the transaction status to 'returned' and refunds the stock_quantity back into the economy.

🚨 5. S.O.S Emergency Board

GET /sos & GET /sos/:id

What it does: Retrieves open emergency requests.

Database Action: Fetches SOS_Requests ordered strictly by priority (Emergency > High > Normal) and created_at DESC.

🔒 POST /sos

What it does: Broadcasts an emergency request to the campus.

Database Action: Inserts into SOS_Requests and utilizes an INSERT INTO Notifications SELECT... subquery to alert all users instantly.

🔒 POST /sos/offer

What it does: Allows a lender to offer their item to fulfill an SOS.

Database Action: Temporarily creates a hidden sos_borrow item listing and initiates a pending transaction, awaiting the requester's approval.

💬 6. In-App Messaging (Tavern Chat)

🔒 GET /messages/:itemId/:user1/:user2

What it does: Fetches the isolated chat history between two users for a specific item.

Database Action: Executes a composite SQL filter requiring an exact match of item_id, sender_id, and receiver_id to prevent chat leakage across different quests.

🔒 POST /messages

What it does: Sends a real-time message.

Database Action: INSERT INTO Messages logging the UTC timestamp, and fires a Notification to the receiver.

⭐ 7. Trust Rating System

🔒 POST /ratings

What it does: Submits a 5-star peer review upon transaction completion.

Database Action: 1. Verifies the transaction_id is completed.

2. Inserts the review.

3. Automatically updates the seller's global reliability_score in the Users table by executing: SELECT ISNULL(AVG(CAST(score AS FLOAT)), 5.0).

❤️ 8. Interactive Wishlist

🔒 POST /wishlist/toggle

What it does: Adds or removes an item from the user's wishlist.

Database Action: Checks if the record exists in Wishlist. If yes, executes a DELETE; if no, executes an INSERT.

🔒 GET /users/:userId/wishlist

What it does: Retrieves the user's saved items.

Database Action: Requires JWT validation, then executes an INNER JOIN between Items and

Wishlist filtered by user_id.